Método de Jacobi para valores y vectores propios

Integrantes: Allan Ruben Prieto Badilla Grupo: 6

1 Propósito y fundamento

Este documento explica el método de Jacobi cíclico, su implementación en parte1.py y un ejemplo completo de tamaño 4 × 4. El objetivo es aproximar los valores propios y sus vectores asociados mediante rotaciones ortogonales, sin utilizar funciones que resuelvan directamente el problema de valores propios.

$$A\,v[j] = \lambda[j]\,v[j], \quad v[j] \neq 0$$

Un valor propio λ_j indica el factor por el cual la matriz transforma a su vector propio v_j. Jacobi aproxima una diagonalización: la diagonal de la matriz transformada contiene las aproximaciones de los valores propios y las rotaciones acumuladas proporcionan los vectores correspondientes [1, 2].

$$B0 = A, \quad V0 = I, \quad B \leftarrow J^T B\,J, \quad V \leftarrow V\,J$$

Como J es ortogonal, cada transformación conserva los valores propios. En aritmética exacta se mantiene $B = V^T A\,V$. Al obtener una matriz diagonal D, se cumple $A\,V = V\,D$. Las columnas de V constituyen los vectores propios [1, 2].

2 Condiciones de aplicación

El método utilizado requiere una matriz real, cuadrada y simétrica. La simetría permite obtener valores propios reales y una base ortonormal de vectores propios. No es necesario exigir definición positiva para aplicar Jacobi [2].

En este portafolio se restringe el trabajo a matrices simétricas semidefinidas positivas. Por ello sus valores propios son no negativos; se permiten valores propios cero. Según el enunciado, las funciones asumen estas propiedades y no incorporan verificaciones de simetría o semidefinición positiva [4].

Esta versión de Jacobi aproxima valores y vectores propios. Es diferente del método de Jacobi para resolver sistemas lineales Ax = b.

3 Construcción de una rotación

Seleccionamos dos índices p < q y anulamos las entradas simétricas b_pq y bq_p. La matriz J coincide con la identidad, excepto en el bloque de filas y columnas p y q [3].

$$J \text{ en el bloque } p\ q \ = \begin{pmatrix} c & s \\ -s & c \end{pmatrix}$$

$$c^2 + s^2 = 1$$

Si $b_pq = 0$, omitimos la rotación. En otro caso calculamos los siguientes coeficientes, usando la raíz de menor magnitud para t [3].

$$\tau = (bqq - bpp) / (2\ bpq)$$

$$t = 1 / (\tau + \sqrt{(1 + \tau^2)}) \qquad \text{si } \tau \geq 0$$

$$t = -1 / (-\tau + \sqrt{(1 + \tau^2)}) \qquad \text{si } \tau < 0$$

$$c = 1 / \sqrt{(1 + t^2)}, \quad s = t\ c$$

Cuando τ = 0 se obtiene t = 1 y c = s = 1/√2. En el código, np.hypot(1.0, tau) calcula $\sqrt{(1 + \tau^2)}$ sin formar explícitamente el cuadrado de tau.

Actualización de la matriz y de los vectores

Al desarrollar B nueva = J^T B J se obtienen las siguientes fórmulas. Todos los términos de la derecha representan valores anteriores a la rotación.

$$b'ip = c\ bip - s\ biq, \quad b'iq = s\ bip + c\ biq \quad \text{para } i \neq p, q$$

$$b'pi = b'ip, \quad b'qi = b'iq$$

$$b'pp = bpp - t\ bpq, \quad b'qq = bqq + t\ bpq, \quad b'pq = b'qp = 0$$

$$V'[:,p] = c\ V[:,p] - s\ V[:,q]$$

$$V'[:,q] = s\ V[:,p] + c\ V[:,q]$$

La implementación conserva copias de las columnas anteriores. Así evita calcular la segunda columna con datos que ya fueron modificados. Solo actualiza las filas y columnas afectadas, sin construir una matriz J completa.

4 Barridos y criterio de parada

Utilizamos Jacobi cíclico. Una iteración k corresponde a un barrido completo por los pares p < q. En una matriz de orden n se visitan n(n − 1)/2 pares por barrido; los pares con entrada cero se omiten. Algunas entradas anuladas pueden cambiar en rotaciones posteriores, por lo que se repiten los barridos [1].

Al finalizar cada barrido se extrae la diagonal y se ordena de mayor a menor. El error exigido por el portafolio es el cambio entre estas aproximaciones ordenadas [4].

$$erk = \|\lambda(k) - \lambda(k-1)\|_2, \quad conv = 1 \text{ si } erk < tol;\ 0 \text{ en otro caso}$$

Se utiliza la diagonal inicial ordenada como $\lambda^{(0)}$. El algoritmo termina al cumplir la tolerancia o agotar iterMax. Al retornar, las columnas de V se ordenan con los mismos índices que los valores propios y se normalizan.

5 Pseudocódigo

```
Entrada: A simétrica semidefinida positiva, iterMax y tol
Salida: valores, V, erk, k y conv

B ← copia real de A;  V ← identidad de orden n
valores ← diagonal de B ordenada de mayor a menor
erk ← infinito;  k ← 0;  conv ← 0

Para k = 1, ..., iterMax:
    anteriores ← copia de valores
    Para p = 0, ..., n−2:
        Para q = p+1, ..., n−1:
            Si B[p,q] = 0: continuar con el siguiente par
            Guardar app, aqq y apq
            Calcular τ, t, c y s según la sección 3
            Para cada i distinto de p y q:
                Guardar bip y biq
                B[i,p] ← c*bip − s*biq; B[p,i] ← B[i,p]
                B[i,q] ← s*bip + c*biq; B[q,i] ← B[i,q]
            B[p,p] ← app − t*apq; B[q,q] ← aqq + t*apq
            B[p,q] ← 0; B[q,p] ← 0
            Guardar columnas vp y vq de V
            V[:,p] ← c*vp − s*vq
            V[:,q] ← s*vp + c*vq
    valores ← diagonal de B ordenada de mayor a menor
    erk ← norma euclidiana de (valores − anteriores)
    Si erk < tol: conv ← 1; salir del ciclo

Ordenar diagonal de B y columnas de V con los mismos índices
Normalizar cada columna de V
Retornar valores, V, erk, k, conv
```

6 Ejemplo completo de tamaño 4 por 4

Elegimos una matriz con dos bloques para mostrar todas las rotaciones necesarias sin ocultar los cálculos. Este ejemplo didáctico es independiente de la comparación de tamaño 200 × 200 de la Parte II.

$$A = \begin{pmatrix} 2 & 1 & 0 & 0 \\ 1 & 2 & 0 & 0 \\ 0 & 0 & 3 & 1 \\ 0 & 0 & 1 & 3 \end{pmatrix}$$

A es simétrica. Además, para cualquier vector real x distinto de cero:

$$x^T Ax = (x1+x2)^2 + x1^2 + x2^2 + (x3+x4)^2 + 2x3^2 + 2x4^2 > 0$$

Por tanto, es definida positiva y cumple la condición de semidefinición positiva. Esta justificación se realiza para el ejemplo, fuera del algoritmo.

Primer barrido

Inicializamos B = A y V = I_4. La diagonal ordenada inicial es $\lambda^{(0)} = (3, 3, 2, 2)^T$. Con índices de Python visitamos (0,1), (0,2), (0,3), (1,2), (1,3) y (2,3).

Para (0,1): app = aqq = 2 y apq = 1. Así, τ = 0, t = 1 y c = s = 1/√2. Las entradas diagonales cambian a 2 − 1 = 1 y 2 + 1 = 3. Las columnas de V pasan a $(1, -1, 0, 0)^T/\sqrt{2}$ y $(1, 1, 0, 0)^T/\sqrt{2}$.

$$B\ tras\ la\ primera\ rotación = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 \\ 0 & 0 & 3 & 1 \\ 0 & 0 & 1 & 3 \end{pmatrix}$$

Los pares (0,2), (0,3), (1,2) y (1,3) tienen entradas cero y se omiten. Para (2,3): app = aqq = 3 y apq = 1. Otra vez τ = 0, t = 1 y c = s = 1/√2. Obtenemos las entradas 3 − 1 = 2 y 3 + 1 = 4.

$$B = \text{diag}(1, 3, 2, 4), \quad \lambda(1) = (4, 3, 2, 1)^T$$

$$er1 = \|(4, 3, 2, 1)^T - (3, 3, 2, 2)^T\|2 = \sqrt{2}$$

Segundo barrido

Todas las entradas fuera de la diagonal son cero. No se realiza ninguna rotación y $\lambda^{(2)} = \lambda^{(1)}$. Así, $er_2 = 0 < 10^{-8}$. Con iterMax = 10000 y tol = 10^{-8} se retorna k = 2 y conv = 1: dos barridos, con dos rotaciones efectivas en total.

7 Vectores propios y comprobación

Ordenamos los valores como $\lambda = (4, 3, 2, 1)^T$ y reordenamos sus columnas asociadas. La matriz final de vectores normalizados es:

$$V = (1/\sqrt{2}) \begin{pmatrix} 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & -1 \\ 1 & 0 & 1 & 0 \\ 1 & 0 & -1 & 0 \end{pmatrix}$$

La correspondencia puede comprobarse antes de dividir por √2, pues el escalamiento no altera la ecuación propia:

$$A(0,0,1,1)^T = (0,0,4,4)^T = 4(0,0,1,1)^T$$

$$A(1,1,0,0)^T = (3,3,0,0)^T = 3(1,1,0,0)^T$$

$$A(0,0,1,-1)^T = (0,0,2,-2)^T = 2(0,0,1,-1)^T$$

$$A(1,-1,0,0)^T = (1,-1,0,0)^T$$

Cada columna tiene norma 1 y el producto interno entre columnas diferentes es cero. En aritmética exacta se cumple $V^TV = I_4$ y cada residuo es cero. En punto flotante aparecen pequeñas diferencias de redondeo.

Medidas para validar el resultado

$$r[j] = \|Av[j] - \lambda[j]v[j]\|2, \quad rmax = max[j]\ r[j], \quad erort = \|V^TV - I\|F$$

En la ejecución comprobada, el ejemplo produjo valores [4, 3, 2, 1], $er_k = 0$, k = 2, conv = 1, residuo máximo cercano a $2{,}22 \times 10^{-16}$ y error de ortogonalidad cercano a $4{,}52 \times 10^{-16}$. Las últimas cifras pueden variar según el entorno.

El error de parada mide cambios entre aproximaciones, no el error verdadero de los pares propios. Por ello conv = 1 no sustituye la revisión de residuos. Asimismo, no debe compararse el número de barridos de Jacobi con iteraciones QR como si cada uno tuviera el mismo costo.

8 Referencias

[1] The University of Texas at Austin. ALAFF, sección 11.3.2, Jacobi's method for computing the Spectral Decomposition. https://www.cs.utexas.edu/~flame/laff/alaff/chapter11-Jacobi-method-spectral-decomposition.html

[2] Wang, R. Harvey Mudd College. Jacobi Eigenvalue Algorithm for Symmetric Real Matrices. https://pages.hmc.edu/ruye/MachineLearning/lectures/ch1/node2.html

[3] Anderson, E. (2000). Discontinuous Plane Rotations and the Symmetric Eigenvalue Problem. LAPACK Working Note 150, sección 5. https://www.netlib.org/lapack/lawnspdf/lawn150.pdf

[4] Soto Quirós, J. P. (2026). Portafolio Computacional, CE-1111, II semestre. Bloque 3, páginas 13–18. Material del curso suministrado por el estudiante.